

Advantages of the COMB Macro in COMB-MCM over General Compute-in-Memory Macros

The increasing computational demands of artificial intelligence (AI) tasks, particularly in resource-constrained edge devices, have spurred significant research into energy-efficient computing paradigms. Compute-in-Memory (CIM) has emerged as a promising approach to address the limitations of traditional von Neumann architectures by performing computational operations within or near memory units, thereby mitigating the substantial energy and latency costs associated with data movement.¹ The initial impetus behind the development of CIM macros was to reduce the intensive memory accesses inherent in AI System-on-Chips (SoCs), leveraging the potential for ultra-high computing efficiency.¹ However, despite their inherent advantages, conventional CIM macros still rely on accessing weight data through both on-chip and off-chip memories, a characteristic they share with near-memory computing architectures.¹ This reliance introduces challenges when considering the overall energy efficiency of the SoC. Specifically, the fundamental issue of the "memory wall," where the energy and time required to move data become the dominant factors in system performance, remains a significant bottleneck that even CIM struggles to fully overcome. This suggests that advancements in CIM technology must prioritize minimizing data movement to achieve substantial improvements in efficiency. Furthermore, the constant updating of weights in machine learning models can lead to considerable power consumption and latency, even in smaller networks, as the energy efficiency of the Multiply-Accumulate (MAC) operations within CIM becomes comparable to that of accessing on-chip memory.¹ Additionally, general CIM approaches often employ sparsity optimization techniques that are heavily dependent on the specific algorithm, are coarse-grained in nature, and necessitate dedicated zero-detection blocks, lacking robust power optimization strategies for more granular sparsity patterns.¹ The COMB-MCM architecture, as detailed in the relevant paper, likely presents itself as an innovative solution designed to address these specific limitations inherent in more general CIM implementations.

To appreciate the advancements offered by the COMB macro, it is first necessary to understand the foundational principles of traditional Compute-in-Memory architectures. In general, CIM involves performing Multiply-Accumulate (MAC) operations, which are fundamental to neural network computations, directly within or in close proximity to memory arrays.² This tight integration aims to reduce the distance data needs to travel, thereby saving energy and improving speed. CIM implementations can be broadly categorized into analog and digital approaches. Analog CIM often utilizes resistive RAM (RRAM) or capacitor-based methods to

perform computations in the analog domain, potentially achieving very high energy efficiency.⁴ However, analog approaches can be susceptible to issues like process variations and may have limitations in terms of precision. Digital CIM, conversely, typically employs SRAM-based architectures, integrating standard digital logic within or near the memory array, offering better robustness and flexibility, albeit potentially with lower energy efficiency compared to analog methods.¹ The choice between analog and digital CIM often depends on the specific application requirements and the desired balance between energy efficiency and precision. Despite the potential benefits, traditional CIM architectures face several common drawbacks. One persistent issue is the "memory wall." Even with computation moved closer to memory, the need to access weight data, particularly for updates in learning scenarios or when dealing with large model sizes, can still create a significant bottleneck.¹ The energy expenditure associated with this data movement can negate some of the efficiency gains offered by in-memory computation.¹² Furthermore, the constant updating of weights in machine learning models poses a challenge. The power consumed and the latency incurred during these updates can be substantial, even for relatively small tasks, especially when the energy efficiency of the MAC operations within the CIM unit becomes comparable to the energy required to access the on-chip memory storing the weights.¹ This near parity in energy costs highlights the importance of optimizing not just the computation itself but also the management and movement of weight data. Another limitation of general CIM architectures lies in their ability to effectively exploit sparsity in neural networks. Many traditional sparsity optimization techniques are highly dependent on the specific neural network algorithm being used, often operate at a coarse-grained level (e.g., pruning entire neurons or filters), and require dedicated hardware blocks to detect zero values.¹ These methods often lack the flexibility to efficiently handle fine-grained or arbitrary sparsity patterns, which are increasingly prevalent in compressed and optimized neural networks.¹ Finally, while digital CIM based on SRAM has been extensively studied, its implementation using non-volatile memory (NVM) has received less attention, despite NVM's potential for high storage density and the ability to retain data without power, which could be particularly beneficial for edge devices.¹ However, research into NVM-based digital CIM is ongoing, as evidenced by the proposal of a SONOS-based macro featuring a simplified lookup table multiplier.¹

The COMB-MCM architecture introduces a novel paradigm by employing a Multi-Chiplet Module (MCM) approach instead of a traditional monolithic System-on-Chip (SoC) to address the complexities of modern computing, particularly for edge machine learning applications.¹ This shift towards MCM recognizes the challenges associated with scaling monolithic designs and the potential for improved

yield and cost-effectiveness through the integration of smaller, independently manufactured chiplets. At the heart of this architecture lies the concept of Computing-on-Memory Boundary (COMB), which represents a strategic compromise between traditional in-memory and near-memory computation.¹ This "memory boundary" likely refers to a computational approach where processing units are placed in close proximity to the memory array, but not necessarily fully embedded within it, aiming to strike an optimal balance between the high energy efficiency and parallelism of in-memory computing and the greater flexibility and potentially lower latency of near-memory computing. The primary objective of the COMB technique within the MCM framework is to achieve high energy efficiency at the macro level while minimizing the overall power overhead of the system.¹

The COMB macro offers several key advantages over general Compute-in-Memory macros, making it particularly well-suited for edge machine learning applications. One of the most significant benefits is its enhanced energy efficiency. The COMB-MCM architecture has demonstrated impressive energy efficiency figures, reportedly achieving 46.4 Tera Operations Per Second per Watt (TOPS/W) at an input sparsity of 50%, with even greater efficiency observed at a 90% input sparsity level.¹ These metrics compare favorably to other processing near memory systems. For instance, a system presented by Alibaba and Fudan University achieved 32.9 TOPS/W while computing with INT3 precision.¹ The high TOPS/W values, especially in conjunction with significant input sparsity, suggest that the COMB macro is specifically designed to capitalize on the inherent sparsity often found in the activations of deep learning models, leading to substantial energy savings. Beyond energy efficiency, COMB-MCM also exhibits a reduced system power overhead.¹ This reduction is largely attributed to the architecture's chiplet-based design, which enables the placement of processing units much closer to the memory units compared to traditional monolithic CIM processors.¹ This proximity is crucial as it minimizes the distance data needs to travel, thereby significantly lowering the energy consumed for communication within the system. The focus on reducing communication costs directly addresses the memory wall issue, which plagues many traditional architectures, including some CIM implementations. By strategically placing computation near memory, COMB-MCM optimizes data access and reduces the energy wasted on long-distance data transfers. The Multi-Chiplet Module (MCM) approach inherent in COMB-MCM further enhances system scalability. The ability to integrate multiple chiplets allows for a flexible increase in both processing power and memory capacity as needed.¹ This modularity is particularly advantageous for the evolving landscape of AI, where model sizes and computational demands are constantly growing. Furthermore, the chiplet-based strategy can lead to tangible cost benefits and accelerated design

cycles compared to the development and manufacturing of large, complex monolithic SoCs.¹ Smaller chiplets are generally associated with higher manufacturing yields, contributing to lower overall costs. The design process can also be expedited as individual chiplets can be designed and tested in parallel. The development of such integrated chip systems is often guided by principles like the Optimal Decomposition-Combination Theory, which provides a framework for optimizing the partitioning of functionalities into chiplets and their subsequent integration.¹

When comparing the COMB-MCM architecture with other existing CIM solutions, its strengths become more apparent. For instance, while an 8-bit sparse Transformer accelerator might achieve a higher TOPS/W rating (e.g., 53.8 TOPS/W⁹), the specific workload, sparsity levels, and precision used in that evaluation might differ from those reported for COMB-MCM. Similarly, a floating-point CIM processor might achieve 31.6 TOPS/W¹⁰, but its target application (likely high-performance computing) and precision requirements are different from COMB-MCM's focus on edge computing.¹ This specific focus on edge applications is a key differentiator for COMB-MCM. Edge devices typically have stringent power consumption and form factor requirements, and COMB-MCM's design choices, including the MCM architecture and the COMB macro, appear to be optimized for these constraints. A significant advantage of the COMB macro is its use of bipolar bitwise sparsity optimization.¹ This approach likely allows for a more fine-grained and flexible exploitation of sparsity compared to the coarse-grained methods often employed in general CIM architectures.¹ By operating at the bit level, COMB-MCM can potentially achieve greater reductions in computation and memory access by effectively skipping operations involving zero bits, leading to further energy savings. While other advanced CIM macros exist, such as the DNV-CIM macro that utilizes non-volatile memory and a lookup table multiplier¹, COMB-MCM's likely reliance on SRAM (given its description as a near-memory computing system and the prevalence of SRAM in CIM research⁹) suggests a different set of trade-offs, potentially prioritizing speed and endurance over the non-volatility and density benefits of NVM.

In conclusion, the COMB macro within the COMB-MCM architecture offers distinct advantages over general Compute-in-Memory macros, particularly for edge machine learning applications. Its enhanced energy efficiency, achieved through a combination of the COMB principle and effective exploitation of input sparsity, coupled with the reduced system power overhead facilitated by the MCM design and the proximity of processing to memory, positions it as a superior solution for resource-constrained environments. The scalability and potential cost benefits of the MCM approach further underscore the advantages of COMB-MCM in addressing the evolving

demands of AI at the edge. By tackling the memory wall issue head-on through architectural innovation and fine-grained sparsity optimization, COMB-MCM represents a significant step forward in energy-efficient computing for edge intelligence.

Works cited

1. COMB-MCM: Computing-on-Memory-Boundary NN Processor with Bipolar Bitwise Sparsity Optimization for Scalable Multi-Chiplet-Module Edge Machine Learning | Request PDF - ResearchGate, accessed May 3, 2025, https://www.researchgate.net/publication/359313701_COMB-MCM_Computing-on-Memory-Boundary_NN_Processor_with_Bipolar_Bitwise_Sparsity_Optimization_for_Scalable_Multi-Chiplet-Module_Edge_Machine_Learning
2. TensorCIM: Digital Computing-in-Memory Tensor Processor With Multichip-Module-Based Architecture for Beyond-NN Acceleration | Request PDF - ResearchGate, accessed May 3, 2025, https://www.researchgate.net/publication/381412726_TensorCIM_Digital_Computing-in-Memory_Tensor_Processor_With_Multichip-Module-Based_Architecture_for_Beyond-NN_Acceleration
3. [2010.12861] MARS: Multi-macro Architecture SRAM CIM-Based Accelerator with Co-designed Compressed Neural Networks - arXiv, accessed May 3, 2025, <https://arxiv.org/abs/2010.12861>
4. A Survey of Computing-in-Memory Processor: From Circuit to Application - ResearchGate, accessed May 3, 2025, https://www.researchgate.net/publication/376781477_A_Survey_of_Computing-in-Memory_Processor_From_Circuit_to_Application
5. PIPECIM: Energy-Efficient Pipelined Computing-in-Memory Computation Engine With Sparsity-Aware Technique | Request PDF - ResearchGate, accessed May 3, 2025, https://www.researchgate.net/publication/384546437_PIPECIM_Energy-Efficient_Pipelined_Computing-in-Memory_Computation_Engine_With_Sparsity-Aware_Technique
6. PIPECIM: Energy-Efficient Pipelined Computing-in-Memory Computation Engine With Sparsity-Aware Technique, accessed May 3, 2025, <https://www.computer.org/csdl/journal/si/2025/02/10701033/20GVT3d3mkU>
7. RRAM compute-in-memory hardware for efficient, versatile, and accurate AI inference | Stanford Digital Repository, accessed May 3, 2025, <https://purl.stanford.edu/nx804ww3510>
8. Design memristor-based computing-in-memory for AI accelerators considering the interplay between devices, circuits, and system - SciEngine, accessed May 3, 2025, <https://www.sciengine.com/doi/10.1007/s11432-022-3627-8>
9. Literature on SRAM-based Compute-In-Memory - GitHub, accessed May 3, 2025, <https://github.com/BUAA-CI-LAB/Literatures-on-SRAM-based-CIM>
10. Leveraging Compute-in-Memory for Efficient Generative Model Inference in TPUs - arXiv, accessed May 3, 2025, <https://arxiv.org/html/2503.00461v1>

11. Simulation of a Fully Digital Computing-in-Memory for Non-Volatile Memory for Artificial Intelligence Edge Applications - PMC, accessed May 3, 2025,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10301468/>
12. Increasing AI Energy Efficiency With Compute In Memory - Semiconductor Engineering, accessed May 3, 2025,
<https://semiengineering.com/increasing-ai-energy-efficiency-with-compute-in-memory/>